Write-up: elevenpack.exe – Unpacking a Packed Binary and Recovering a Custom Cipher

1. Initial Observation

The binary `elevenpack.exe` appeared packed: its entry point was small, and the initial instructions did not resemble standard compiler output. Static analysis with IDA Pro showed only a few API calls and no clear logic. Therefore, dynamic analysis was required to unpack the binary before meaningful reverse engineering could begin.

2. Unpacking with x64dbg

2.1 Setting a Breakpoint on `ExitProcess`

The executable was loaded into x64dbg. A breakpoint was set on `ExitProcess` – a common technique because many unpackers call `ExitProcess` after they have restored the original code. When the breakpoint was hit, the call stack was examined.

2.2 Locating the Original Entry Point (OEP)

The call stack revealed a return address that pointed to a memory region not belonging to the packer stub. Navigating to that address showed a typical function prologue (e.g., `push rbp`, `sub rsp, 80h`) – this was the **Original Entry Point (OEP)** of the unpacked program.

2.3 Dumping the Unpacked Binary

Using x64dbg's built-in memory dump feature (or the Scylla plugin), the entire process memory was dumped and saved as a new executable. This dumped file was then loaded into IDA Pro for static analysis.

3. Static Analysis with IDA Pro

3.1 Recovered Code Structure

The unpacked binary contained a clear `start` function (the full decompilation is shown below). Key components are highlighted:

```
__int64 start()
{
  // ... variable declarations ...
  v0 = sub_140004858(1u);
  print(&unk_140012370, v0);                    // prints "> "
  v1 = sub_140004858(1u);
  sub_140004CE0(v1);                            // possibly flushes output
  // zero out a 64-byte buffer
  *(_OWORD *)v20 = 0i64;
  v21 = 0i64;
  v22 = 0i64;
  v23 = 0i64;
  v2 = sub_140004858(0);
  sub_140004EB0((__int64)v20, 64i64, (__int64)v2); // read user input (max 64 bytes)
  // remove trailing newline/carriage return
  for ( i = sub_140011160((__int64)v20); i; *((_BYTE *)v20 + i) = 0 )
  {
    n10 = *((_BYTE *)&v19[3] + i + 7);
    if ( n10 != 10 && n10 != 13 ) break;
    --i;
  }
  // take first 5 bytes as key
  i_1 = 0i64;
  v25 = 0;          // key storage (5 bytes)
  v26 = 0;
  do {
    if ( i_1 >= i ) break;
    *((_BYTE *)&v25 + i_1) = *((_BYTE *)v20 + i_1);
    ++i_1;
  } while ( i_1 < 5 );

  // main decryption loop - processes 32 bytes in blocks of 8
  v6 = 1;
  v24 = 1;
  do {
    // compute block-dependent constants
    v7 = 27 * (v6 - 1);
    v8 = ((v6 - 1) & 3) + 1;
    v9 = (v6 & 3) + 1;
    v10 = -98 * (v6 - 1);
    v11 = 7 * (v6 - 1);
    // index 0 of this block
    *((_BYTE *)v19 + v6 - 1) = byte_140012350[v6 - 1] ^ (v11
        + ((v10 + 55) ^ __ROL1__(*((_BYTE *)&v25 + (v6 - 1) % 5) ^ (v7 - 91), v8)));
    // index 1
    *((_BYTE *)v19 + v6) = byte_140012350[v6] ^ (v11
```

```
        + ((v10 - 43) ^ __ROL1__(*((_BYTE *)&v25 + v6 % 5) ^ (v7 - 64), v9)) + 7);
    // ... similar assignments for indices 2..7 of the block ...
    v6 = v24 + 8;
    v24 += 8;
  } while ( (unsigned int)(v15 + 7) < 0x20 ); // loop until 32 bytes processed

  // output the 32-byte result
  v16 = sub_140004858(1u);
  sub_140005664(v19, 1i64, 32i64, v16);
  v17 = sub_140004858(1u);
  sub_140005018(10i64, v17);                    // print newline
  return 0i64;
}
```

The constant ciphertext is stored in `.rdata`:

```
.rdata:0000000140012350 byte_140012350  db 0D8h, 0CAh, 7Bh, 0DAh, 48h, 1Ch, 0EAh, 0EEh, 42h,
0A4h
.rdata:000000014001235A                 db 0F4h, 0D3h, 0EEh, 0A6h, 0D1h, 74h, 0BCh, 4Dh, 0AAh
.rdata:0000000140012363                 db 0A5h, 3Eh, 0BFh, 0C1h, 6Ah, 57h, 80h, 0B5h, 0A8h,
3Eh
.rdata:000000014001236D                 db 0F5h, 5Dh, 20h
```

3.2 Understanding the Algorithm

The `start` function implements a custom cipher that transforms a 5-byte key into a 32-byte output (the flag). Each output byte is computed as:

```
output[i] = cipher[i] XOR f(i, key[...])
```

where `f` involves:

- additions (e.g., `7*(v6-1)`, `-98*(v6-1)`, `27*(v6-1)`)
- XOR with constants (like `55`, `-43`, `115`, etc.)
- an **8-bit rotate left** (`__ROL1__`)

The rotate operation is implemented in assembly using shifts and ORs. For example, the first assignment produces the following assembly snippet:

```asm
.text:0000000140001122     mov     ecx, r14d          ; shift count (v8)
.text:0000000140001125     movzx   edx, r9b           ; value to rotate
.text:0000000140001129     shl     dl, cl
.text:000000014000112B     movzx   ecx, r12b
.text:000000014000112F     shr     r9b, cl
.text:0000000140001132     or      dl, r9b            ; ROL result
```

Because the algorithm processes the 32 bytes in blocks of 8, and because the first block (indices 0-4) depends on **distinct** key bytes (one per output position), we can recover the key directly by assuming the first 5 output bytes are known.

3.3 Recovering the Key via Known Plaintext

We assume the final output is a flag in the common CTF format `FLAG{...}`. Therefore, the first five output bytes are:

Index	Expected char	ASCII
0	'F'	70
1	'L'	76
2	'A'	65
3	'G'	71
4	'{'	123

For index 0, the decompiled code gives:

```
v19[0] = byte_140012350[0] ^ ( 55 ^ ROL1(key[0] ^ (0 - 91), 1) )
```

Since `0 - 91 = -91 ≡ 165 (mod 256)`, and `byte_140012350[0] = 0xD8 = 216`, we solve:

```
70 = 216 ^ K    →    K = 216 ^ 70 = 158
158 = 55 ^ ROL1(key[0] ^ 165, 1)
ROL1(key[0] ^ 165, 1) = 55 ^ 158 = 169
```

```
key[0] ^ 165 = ROR1(169, 1) = 212
key[0] = 212 ^ 165 = 113  →  'q'
```

Similarly for indices 1-4 (using their respective formulas from the decompiled code):

- Index 1: `key[1] = 106` → `'j'`
- Index 2: `key[2] = 48` → `'0'`
- Index 3: `key[3] = 111` → `'o'`
- Index 4: `key[4] = 77` → `'M'`

Thus the key is `qj0oM`.

3.4 Decrypting the Full Flag

Using the recovered key, we run the same decryption routine (e.g., with a Python script) to compute all 32 output bytes. The result is:

```
FLAG{GF3yuOKnMzBefXqD5N58ptjh0}
```

All characters are printable and follow the expected flag syntax, confirming the correctness of the key.

4. Conclusion

By dynamically unpacking the binary with x64dbg (breakpoint on `ExitProcess` + memory dump) and statically analyzing the recovered code in IDA Pro, we identified a custom cipher. Assuming the flag prefix `FLAG{` gave five independent equations that directly yielded the 5-byte key. The full flag was then decrypted.

Final Flag: `FLAG{GF3yuOKnMzBefXqD5N58ptjh0}`